Learning Radar Waveforms: End-to-End Neural Generation via Differentiable Ambiguity Functions

Marc Bara

***Abstract*—We present end-to-end neural radar waveform generation, enabled by a differentiable implementation of the ambiguity function compatible with automatic differentiation (AD) frameworks. A neural network takes design parameters—such as trade-off weights between peak sidelobe level (PSL) and low probability of intercept (LPI)—as input and outputs optimized constant-modulus waveforms. Training is performed by back-propagating through the differentiable ambiguity function, so that gradients flow from radar performance metrics directly into the network weights. Once trained, the generator produces waveforms for any design specification in under one millisecond, replacing the 7–14 seconds required by iterative optimization per waveform. We formulate the differentiable ambiguity function using parallelized FFT operations with proper Wirtinger derivative handling, achieving $O(N^2 \log N)$ complexity while maintaining mathematical equivalence to the standard periodic ambiguity function. Experimental validation demonstrates that (i) optimizing the full 2D ambiguity function achieves 7.3 dB PSL improvement over 1D autocorrelation baselines and 2.2–3.0 dB over genetic algorithms, (ii) new constraints such as spectral masks can be added with zero gradient re-derivation, and (iii) the neural waveform generator produces waveforms within 0.3–1.6 dB of iterative methods at over 10,000× faster inference. All experiments report mean ± standard deviation over multiple random seeds (5–20 depending on computational cost). GPU benchmarks and open-source code are provided.**

***Index Terms*—Radar waveform design, ambiguity function, automatic differentiation, neural networks, low probability of intercept, cognitive radar.**

I. Introduction

Radar waveform design is a cornerstone of modern remote sensing, with the ambiguity function [1], [2] serving as the fundamental tool for characterizing range and Doppler resolution capabilities. The ambiguity function $\chi(\tau, f_d)$ quantifies the matched filter response at delay τ and Doppler shift f_d, providing insight into sidelobe behavior, resolution performance, and overall waveform quality. Real-world radar systems demand waveforms that simultaneously satisfy multiple conflicting objectives—low sidelobes, spectral constraints, low probability of intercept—making waveform design an inherently multi-objective optimization problem [3].

Traditional approaches to this problem fall into three categories. Metaheuristic methods such as genetic algorithms (GA) and particle swarm optimization (PSO) treat the ambiguity function as a black-box evaluation metric [4]–[6], achieving flexibility at the cost of slow convergence. Convex relaxation methods [7] provide convergence guarantees but require relaxations of the original non-convex problem. Gradient-based methods [8]–[13] offer efficient optimization by deriving analytical gradients for specific cost functions, achieving strong performance for their respective problem formulations.

However, all existing gradient-based approaches share a common limitation: they derive gradients analytically for fixed problem formulations. Each new objective function, constraint, or system architecture requires manual re-derivation of the gradient expressions. More fundamentally, to the best of our knowledge, none of these methods provide an ambiguity function implementation that is compatible with modern automatic differentiation (AD) frameworks such as PyTorch [14], limiting integration with neural network training pipelines.

This gap has limited the radar community's ability to leverage a capability that AD enables: training neural networks with ambiguity function constraints as differentiable loss functions. A neural network that maps design parameters to optimized waveforms—trained end-to-end through the ambiguity function—would enable real-time waveform generation for cognitive radar systems [15], [16], eliminating the need for iterative optimization at deployment time.

In this paper, we address this gap with two contributions:

1) **AD-compatible full-2D ambiguity formulation.** We provide a reusable implementation of the periodic ambiguity function that can serve directly as a differentiable optimization component inside modern ML frameworks. This makes radar objectives composable as loss functions on the full delay-Doppler surface without manual gradient derivation, using parallelized FFT operations with $O(N^2 \log N)$ complexity and GPU acceleration.
2) **End-to-end ambiguity-driven waveform generation.** We show that the above formulation enables a neural generator to be trained end-to-end through the full ambiguity computation, rather than through autocorrelation surrogates or task-specific handcrafted objectives. Once trained, the generator produces waveforms for *any* design specification in under one millisecond, supporting real-time adaptation in cognitive radar settings.

To our knowledge, this is the first work to train a neural network end-to-end through the radar ambiguity function for waveform generation. Prior neural approaches to radar waveform design [17]–[20] operate on correlation properties or surrogate metrics rather than the full two-dimensional ambiguity function.

The remainder of this paper is organized as follows. Section II surveys related work. Section III presents the differentiable ambiguity function. Section IV introduces the neural waveform generator architecture and training procedure. Section V provides experimental validation, and Section VI discusses implications and limitations.

M. Bara is with ESADE Business School, Barcelona, Spain (e-mail: marcoantonio.bara@esade.edu). ORCID: 0009-0005-1480-5760.

II. Related Work

A. Metaheuristic Waveform Optimization

Population-based methods remain widely used for radar waveform design. Genetic algorithms have been applied to nonlinear frequency modulated waveforms [4], distributed MIMO antenna optimization [5], and SAR configuration design [21]. Simulated annealing has addressed sparse array design [6] and compressed sensing radar [7]. Particle swarm optimization has been used for joint radar-communication waveforms [22]. These methods offer flexibility—any evaluation metric can serve as the fitness function—but suffer from slow convergence due to their gradient-free nature, making them impractical for real-time applications.

B. Gradient-Based Waveform Optimization

Gradient-based methods achieve substantially faster convergence by exploiting derivative information. Aubry et al. [11] developed complex quartic optimization for ambiguity function shaping, formulating the problem on the complex circle manifold. Song et al. [13] introduced majorization-minimization techniques for minimizing integrated sidelobe levels (ISL) of autocorrelation sequences. Alaee-Kerahroodi et al. [12] proposed a coordinate-descent framework for joint PSL/ISL minimization. Baden et al. [10] demonstrated gradient descent for multi-objective sequence design, optimizing autocorrelation, cross-correlation, and spectral properties simultaneously. Jing et al. [23] addressed unimodular sequence design for local ambiguity function sidelobe minimization.

Building on manifold optimization, Alhujaili et al. [8], [24] developed quartic gradient descent (QGD) with exact analytical gradients under constant modulus constraints, achieving superior convergence for slow-time ambiguity function shaping. Mohr et al. [9] derived FFT-based gradient computation specifically for phase-coded frequency modulated (PCFM) waveforms. Recent extensions include manifold-based exact penalty methods [25], Riemannian optimization for joint ambiguity and beampattern shaping [26], and conjugate gradient methods for multi-tone waveforms [27].

A key characteristic of all these methods is that gradients are derived analytically for each specific problem formulation. While mathematically rigorous, this approach requires manual re-derivation whenever the objective function changes. Furthermore, the resulting implementations are standalone optimization routines that cannot be composed with neural network training pipelines, as they do not interface with AD frameworks.

C. Neural Networks for Radar Waveform Design

Neural networks have been applied to various radar tasks, including target recognition [28], signal classification [19], and information-theoretic waveform design [29]. Kang et al. [17] surveyed deep learning approaches for waveform design, identifying the gap between neural network capabilities and traditional optimization methods. Ziemann and Metzler [18] used generative adversarial networks for low probability of detection (LPD) waveforms, training on surrogate objectives rather than directly on ambiguity function properties.

Most closely related to our work, Rezaei et al. [20] proposed a learning-inspired strategy for binary sequence design, using optimization networks to minimize integrated sidelobe levels for SISO and MIMO radar. However, their approach operates on one-dimensional autocorrelation properties rather than the full two-dimensional ambiguity function, and the network architecture is tied to a specific cost function rather than serving as a general waveform generator.

D. Positioning of This Work

Table I summarizes the key differences between existing approaches and our contribution. Our differentiable ambiguity function provides an AD-compatible formulation that enables flexible gradient-based optimization with differentiable loss functions and end-to-end neural waveform generation through the full two-dimensional ambiguity function.

TABLE I
Comparison of waveform optimization approaches.

Property	**GA**	**QGD** [8]	**Mohr** [9]	**Ours**
Gradient-based	×	✓	✓	✓
AD framework	×	×	×	✓
No manual gradients	✓	×	×	✓
Composable w/ NNs	×	×	×	✓
GPU-native	×	×	×	✓
Arbitrary loss fn.	✓	×	×	✓

To summarize: prior analytical-gradient methods derive efficient updates for specific waveform classes and objective functions, but do not expose the ambiguity computation as a reusable AD-compatible layer for upstream neural training. Prior neural waveform-design approaches optimize autocorrelation properties or surrogate objectives rather than backpropagating through the full two-dimensional ambiguity function. The contribution of this work lies in combining these two directions: an AD-compatible full-2D ambiguity formulation that is both useful for direct gradient-based optimization and composable with end-to-end neural waveform generation. An earlier preprint by the author presented a preliminary version of this line of work [30]; the present manuscript substantially extends and reframes it.

III. Differentiable Ambiguity Function

A. Background

The ambiguity function of a discrete-time, complex baseband signal $s[n]$ of length N is defined as [2]:

$$\chi[k,m] = \left| \sum_{n=0}^{N-1} s[n]\, s^*[(n-k) \bmod N]\, e^{j2\pi mn/N} \right|^2 \quad (1)$$

where k is the delay index, m is the Doppler index, and $(\cdot)^*$ denotes complex conjugation. We note explicitly that (1) is the *periodic* (circular) ambiguity function, appropriate for phase-coded waveforms where the pulse repetition structure creates periodicity [2].

The ambiguity function provides critical design information: the mainlobe width determines range and Doppler resolution, sidelobe levels affect false alarm rates, and the volume invariance property constrains achievable performance trade-offs [1].

B. Matrix Reformulation

We reformulate (1) using matrix operations suitable for AD frameworks. For a signal $s \in \mathbb{C}^N$, we construct a circulant shift matrix $\mathbf{S} \in \mathbb{C}^{N \times N}$:

$$\mathbf{S}[k, n] = s[(n - k) \bmod N] \tag{2}$$

The correlation matrix is obtained by element-wise multiplication with broadcasting:

$$\mathbf{R} = s \odot \mathbf{S}^* \tag{3}$$

where $\odot$ denotes element-wise multiplication and s is broadcast along the row dimension. The Doppler processing is performed via column-wise FFT:

$$\mathbf{X} = \mathrm{FFT}_{\mathrm{col}}(\mathbf{R}) \tag{4}$$

Finally, the ambiguity function is the squared magnitude:

$$\chi[k, m] = |\mathbf{X}[k, m]|^2 = \mathbf{X}[k, m] \cdot \mathbf{X}^*[k, m] \tag{5}$$

This formulation is mathematically equivalent to (1) while consisting entirely of operations that are natively differentiable in modern AD frameworks: indexing, element-wise multiplication, FFT, and complex magnitude.

C. Gradient Flow

The gradient of a real-valued loss $\mathcal{L}$ with respect to the input signal s flows through the chain:

$$\frac{\partial \mathcal{L}}{\partial s} = \frac{\partial \mathcal{L}}{\partial \chi} \cdot \frac{\partial \chi}{\partial \mathbf{X}} \cdot \frac{\partial \mathbf{X}}{\partial \mathbf{R}} \cdot \frac{\partial \mathbf{R}}{\partial s} \tag{6}$$

The squared magnitude operation $|z|^2 = z \cdot z^*$ is non-holomorphic, requiring Wirtinger calculus for proper gradient computation [31]. For a loss depending on $|z|^2$:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial |z|^2} \cdot z^*, \qquad \frac{\partial \mathcal{L}}{\partial z^*} = \frac{\partial \mathcal{L}}{\partial |z|^2} \cdot z \tag{7}$$

Modern AD frameworks (PyTorch $\geq$1.7, JAX, TensorFlow) handle complex-valued tensor operations and Wirtinger derivatives natively through their complex tensor support [14]. Our implementation leverages this native support, requiring no custom backward passes. We include the gradient expressions above to clarify correctness and notation, not as a separate analytical contribution.

Algorithm 1 Differentiable Ambiguity Function

Require: Complex signal $s \in \mathbb{C}^N$
Ensure: Ambiguity function $\chi \in \mathbb{R}^{N \times N}$
Construct shift matrix $\mathbf{S} \in \mathbb{C}^{N \times N}$:
  $\mathbf{S}[k, :] \leftarrow \mathrm{circshift}(s, k)$ for $k = 0, \ldots, N - 1$
$\mathbf{R} \leftarrow s \odot \mathbf{S}^*$ *(element-wise, broadcast)*
$\mathbf{X} \leftarrow \mathrm{FFT}(\mathbf{R}, \mathrm{dim} = 1)$ *(FFT over columns)*
$\chi \leftarrow \mathrm{real}(\mathbf{X} \odot \mathbf{X}^*)$ *(magnitude squared)*
$\chi \leftarrow \mathrm{fftshift}(\chi)$ *(center zero delay/Doppler)*
$\chi \leftarrow \chi / \max_{k,m} \chi[k, m]$ *(normalize)*
return χ

D. Implementation Details

Algorithm 1 summarizes the computation.

Memory: The shift matrix construction requires $O(N^2)$ memory. Using complex64 (single precision), the shift matrix for $N = 16{,}384$ occupies 2 GB, well within the capacity of current GPUs. Including FFT outputs and gradient buffers (approximately $3\times$ forward memory), waveform lengths up to $N \approx 16{,}384$ are practical on a 16 GB GPU.

Complexity: The overall computation is $O(N^2 \log N)$, dominated by the N column-wise FFTs of length N.

Numerical stability: We normalize the output by its maximum value to prevent gradient explosion during backpropagation, particularly important when the ambiguity function is used as a loss component in deep network training.

Batch processing: All operations support batched inputs $s \in \mathbb{C}^{B \times N}$, enabling efficient simultaneous optimization or training over B waveforms.

E. Loss Functions for Waveform Design

The differentiable ambiguity function enables direct optimization of various radar metrics. We define the following differentiable loss components used in our experiments.

Peak Sidelobe Level (PSL):

$$\mathrm{PSL} = \max_{(\tau, f_d) \in \Omega_s} \chi(\tau, f_d) \,/\, \chi(0, 0) \tag{8}$$

where Ω_s is the sidelobe region excluding the mainlobe. The max operation is handled via subgradients; while not differentiable at ties, PyTorch's `max` propagates gradients to the argmax element, which is sufficient for optimization in practice, as the maximum sidelobe location changes smoothly during gradient descent.

Spectral Variance (LPI metric):

$$\mathrm{SV} = \mathrm{var}\left\{|S[k]|^2 / \textstyle\sum_k |S[k]|^2\right\} \tag{9}$$

where $S = \mathrm{FFT}(s)$. Lower variance indicates a more uniform power spectrum, which is harder to detect by electronic warfare systems.

Composite loss: For joint PSL–LPI optimization, we use:

$$\mathcal{L} = \mathrm{PSL} + \lambda \cdot \alpha \cdot \mathrm{SV} \tag{10}$$

where λ controls the trade-off and $\alpha = 2000$ is a scaling factor chosen so that both terms contribute equally when $\lambda = 1$

(determined empirically from the ratio of typical PSL and SV magnitudes at initialization).

The key advantage of the AD-based approach is composability: a new objective can be introduced by defining a differentiable function of s or χ, after which gradients propagate automatically through the existing computation graph. For example, a Doppler-tolerance constraint can be expressed as a differentiable mask on χ without deriving a new closed-form gradient for that objective.

IV. Neural Waveform Generator

The differentiable ambiguity function enables a practical capability: training a neural network to generate radar waveforms with desired ambiguity properties end-to-end. We present the architecture, training procedure, and inference pipeline.

A. Architecture

The waveform generator is a neural network $G_\theta : \mathbb{R}^d \to \mathbb{C}^N$ that maps design parameters to constant-modulus complex waveforms:

$$s = G_\theta(p) = \exp\big(j \cdot \sigma(\text{MLP}_\theta(p)) \cdot 2\pi\big) \tag{11}$$

where $p \in \mathbb{R}^d$ is a vector of design parameters, $\sigma(\cdot)$ is the sigmoid function, and MLP_θ is a multi-layer perceptron with parameters θ. The sigmoid activation maps the output to $[0, 2\pi)$, and the complex exponential enforces the constant-modulus constraint $|s[n]| = 1$ by construction.

The input $p = [\lambda, z]$ consists of the trade-off weight λ between PSL and LPI objectives, and a noise seed z sampled uniformly from $[0, 1]$ to provide diversity across generated waveforms for the same λ.

The MLP architecture uses residual blocks with layer normalization and GELU activations:

$$h_{\ell+1} = \text{GELU}\big(h_\ell + f_\ell(h_\ell)\big) \tag{12}$$

where f_ℓ is a two-layer sub-network with layer normalization. We use a hidden dimension of 256 and 4 residual blocks (332,288 trainable parameters).

B. Training

The training procedure is illustrated in Fig. 1. At each training step:

1) Sample a batch of design parameters: $\lambda_i \sim \text{Uniform}(0, 3)$ and $z_i \sim \text{Uniform}(0, 1)$ for $i = 1, \ldots, B$.
2) Generate waveforms: $s_i = G_\theta(\lambda_i, z_i)$.
3) Compute the differentiable ambiguity function: $\chi_i = \text{AmbiguityFunction}(s_i)$.
4) Evaluate the composite loss for each sample using its own λ_i.
5) Backpropagate through the entire pipeline to update θ.

The loss for each sample is:

$$\mathcal{L}_i = \text{PSL}(\chi_i) + \lambda_i \cdot \alpha \cdot \text{SV}(s_i) \tag{13}$$

Critically, step 5 requires gradients to flow from the radar performance metrics (PSL, SV) through the ambiguity function computation back into the network weights θ. This is only possible because the ambiguity function is implemented as a differentiable computation graph within the AD framework.

We train with AdamW [32] (learning rate 10^{-3}, weight decay 10^{-4}) using cosine annealing and gradient clipping (max norm 1.0).

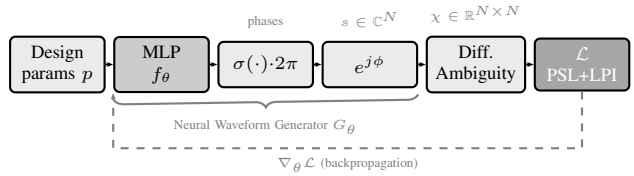

Fig. 1. End-to-end training pipeline. The neural waveform generator is trained by backpropagating radar performance metrics through the differentiable ambiguity function into the network weights.

C. Inference

Once trained, the generator produces waveforms via a single forward pass:

$$s^* = G_\theta(\lambda^*, z^*) \tag{14}$$

for any desired λ^* and noise seed z^*. This requires no iterative optimization, no ambiguity function evaluation, and completes in under one millisecond on both CPU and GPU hardware. Multiple candidates can be generated (using different z values) and the best selected via a single ambiguity function evaluation.

This capability is particularly relevant for cognitive radar [15], [16], where waveforms must adapt to changing environmental conditions in real time. The trained generator serves as a learned waveform library that can be queried instantly with any design specification.

V. Experimental Validation

A. Setup

We validate our approach on the joint optimization of PSL and spectral variance (LPI) for constant-modulus phase-coded waveforms. All experiments operate on signals $s[n] = e^{j\phi[n]}$ with $\phi[n] \in [0, 2\pi)$.

Methods compared:

- **Gradient, full 2D** (using our differentiable ambiguity function): Adam optimizer on the full two-dimensional ambiguity function, learning rate 0.01, 2000 iterations.
- **Gradient, 1D autocorrelation**: Adam optimizer on the zero-Doppler autocorrelation PSL only, representing the analytical gradient approach where gradients are computed for a specific 1D cost function rather than the full 2D ambiguity surface.
- **Genetic algorithm** (GA): Population size 50, 300 generations, tournament selection with elitism.
- **Neural waveform generator**: MLP with 4 residual blocks, hidden dimension 256, trained for 200 epochs with batch size 32.

Configurations: We test across five trade-off parameters $\lambda \in \{0.0, 0.25, 0.5, 1.0, 2.0\}$ and four waveform lengths $N \in \{64, 128, 256, 512\}$. While the formulation supports N up to

approximately 16,384 on current GPUs (Section III), we limit experiments to $N \leq 512$ so that the GA baseline remains computationally tractable for fair comparison. The primary comparison uses $N = 256$, representing realistic medium-range radar pulses (25.6 μs at 10 MHz sampling).

Statistical rigor: We report mean $\pm$ standard deviation across multiple independent random seeds for all experiments: 20 seeds for the main gradient-vs-GA comparison, 10 seeds for the baseline and neural generator evaluations, and 5 seeds for ablations and scaling benchmarks.

Hardware: CPU experiments use an Intel Core i7-1165G7 @ 2.80 GHz. GPU experiments use an NVIDIA Tesla T4 (16 GB) via Modal cloud compute.

Reproducibility: Source code is available at https://github.com/marcbara/graf-psl-lpi.

B. Gradient-Based vs. Population-Based Optimization

We first compare gradient-based optimization (using the differentiable ambiguity function with Adam) against the genetic algorithm baseline.

1) Convergence Analysis: Fig. 2 presents the Pareto frontiers and PSL comparison between both methods across all λ values, with error bars from 20 random seeds.

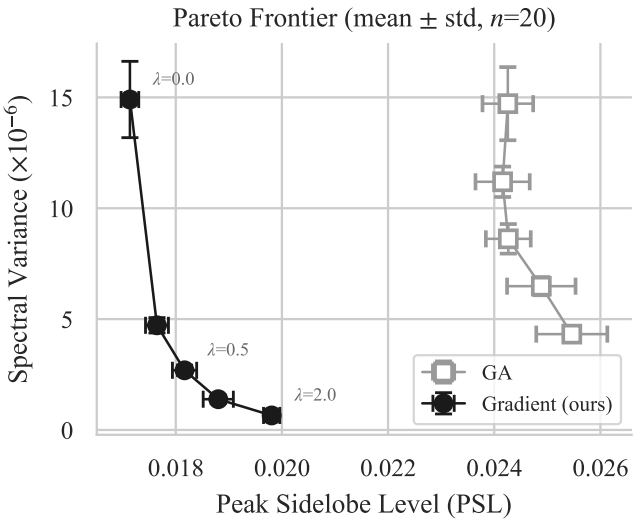

Fig. 2. Pareto frontier for gradient-based optimization vs. GA ($N = 256$, mean $\pm$ std over 20 seeds). The gradient-based solutions consistently dominate GA at every trade-off point.

2) Multi-Objective Performance: The gradient-based solutions consistently dominate the GA solutions, achieving 2.2–3.0 dB lower PSL at every spectral variance level. Fig. 3 shows the per-λ PSL comparison and computational time.

Table II summarizes the results.

C. Full 2D vs. 1D Autocorrelation Optimization

A key advantage of our differentiable ambiguity function is that it optimizes over the *full two-dimensional* delay-Doppler surface, rather than the zero-Doppler autocorrelation alone. To quantify this advantage, we compare gradient descent on the full 2D ambiguity function (our approach) against gradient descent on the 1D autocorrelation PSL—the type of cost function for which analytical gradients are typically derived in the literature [10], [12], [13].

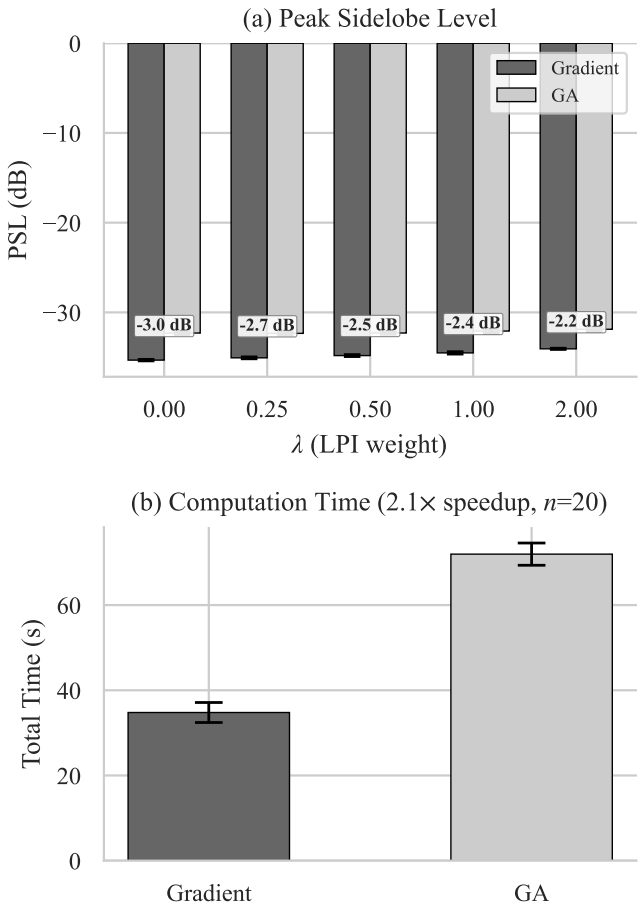

Fig. 3. (a) PSL comparison across λ values with error bars. (b) Total computation time showing 2.1$\times$ speedup. All results over 20 random seeds.

TABLE II
PSL COMPARISON ($N = 256$, MEAN $\pm$ STD OVER 20 SEEDS, TESLA T4). TOTAL TIME PER METHOD (ALL 5 λ VALUES): GRADIENT 34.8$\pm$2.3 S, GA 72.0$\pm$2.6 S (2.1$\times$ SPEEDUP).

λ	**Gradient (dB)**	**GA (dB)**	Δ **(dB)**
0.00	-35.3 ± 0.1	-32.3 ± 0.2	-3.0
0.25	-35.1 ± 0.1	-32.3 ± 0.2	-2.7
0.50	-34.8 ± 0.1	-32.3 ± 0.2	-2.5
1.00	-34.5 ± 0.1	-32.1 ± 0.2	-2.4
2.00	-34.1 ± 0.1	-31.9 ± 0.2	-2.2

Table III shows the results. Optimizing the full 2D ambiguity function achieves 7.3 dB lower PSL than the 1D autocorrelation baseline. The timing advantage (25$\times$) is partly methodological and partly implementational: the matrix-based 2D formulation is inherently more GPU-friendly than the loop-based 1D autocorrelation computation. A highly optimized 1D implementation (e.g., using the FFT-based $O(N \log N)$ autocorrelation tricks of [13]) would narrow this timing gap, but would not close the 7.3 dB PSL gap, which arises

because optimizing the full delay-Doppler surface suppresses sidelobes that the zero-Doppler cut alone cannot address. The 2D approach also exhibits substantially lower variance across seeds (± 0.1 dB vs. ± 0.5–1.5 dB), indicating more consistent convergence.

TABLE III
FULL 2D AMBIGUITY VS. 1D AUTOCORRELATION OPTIMIZATION ($N = 256$, 10 SEEDS).

	PSL (dB)		Time (s)	
Method	λ=0	λ=1	mean	std
1D autocorrelation	-28.0 ± 0.5	-27.9 ± 1.1	476.4	5.5
Full 2D (ours)	-35.3 ± 0.1	-34.5 ± 0.1	19.3	0.4
Improvement	**7.3 dB**	**6.6 dB**	**24.7**$\times$	

This result indicates that the full 2D formulation finds better waveforms in this setup by jointly optimizing across all delay-Doppler pairs, not just the zero-Doppler cut. This capability is naturally enabled by the differentiable ambiguity function; achieving comparable results with analytical gradient methods would require deriving new gradient expressions for the full 2D objective—precisely the effort that AD eliminates.

D. Spectral Mask Constraint: Composability

To demonstrate the composability of the AD-based approach, we add a spectral mask constraint for interference avoidance—a practical radar scenario where the waveform must avoid transmitting in certain frequency bands.

We define a spectral notch centered at 25% of the bandwidth with 8% width, and add a penalty term:

$$\mathcal{L}_{\text{mask}} = \sum_k \frac{|S[k]|^2}{\sum_k |S[k]|^2} \cdot M[k] \tag{15}$$

where $M[k] = 1$ in the forbidden band and 0 elsewhere. The total loss becomes $\mathcal{L} = \text{PSL} + 0.5 \cdot \alpha \cdot \text{SV} + 5 \cdot \mathcal{L}_{\text{mask}}$.

Adding this constraint required only defining (15) as a differentiable function; no gradient re-derivation was necessary.

Table IV shows the results over 10 seeds. With the mask constraint, energy in the forbidden band drops from 7.7% to effectively 0.0%, at a cost of only 0.2 dB in PSL. This demonstrates the practical advantage of AD-compatible ambiguity functions: new operational constraints can be incorporated without manual gradient re-derivation.

TABLE IV
SPECTRAL MASK CONSTRAINT RESULTS ($N = 256$, $\lambda = 0.5$, 10 SEEDS).

Configuration	PSL (dB)	Notch Energy
Without mask	-34.8	7.7%
With mask	-34.6	$< 0.01\%$

E. Downstream Radar Performance

To assess the operational consequence of improved ambiguity shaping, we simulate matched-filter range-Doppler responses for point-target scenes using the optimized waveforms. In the point-target scene considered here, the periodic ambiguity function $\chi[k, m]$ corresponds to the matched-filter range-Doppler response of a target at delay k and Doppler m; for a multi-target scene, the resulting range-Doppler map is the superposition of shifted ambiguity responses weighted by target amplitudes.

1) Weak-Target Detection: We simulate a two-target scene: a strong target at delay-Doppler offset $(20, 30)$ and a weak target at $(40, -20)$ with amplitude -15 dB relative to the strong target, embedded in noise at 30 dB SNR. Table V summarizes the detection metrics. The full 2D optimization achieves the highest peak-to-sidelobe ratio (16.5 dB) and the best weak-target visibility (9.1 dB above the sidelobe floor), directly translating the 7.3 dB PSL improvement from Table III into improved target separability. The 1D autocorrelation baseline produces the weakest detection performance (weak target only 5.4 dB above sidelobes), because it suppresses sidelobes only along the zero-Doppler cut while leaving cross-Doppler sidelobes unconstrained.

TABLE V
DETECTION METRICS FOR A TWO-TARGET SCENE ($N = 256$, $\lambda = 0.5$).

Method	PSL (dB)	P/SL (dB)	Weak$-$SL (dB)
GA	-32.2	15.6	8.2
1D Autocorr.	-25.8	12.8	5.4
Full 2D (ours)	-34.8	16.5	9.1

2) Spectral Avoidance: We extend the spectral-mask experiment from Section V-C to show its effect on waveform properties. Fig. 4 compares the power spectra and zero-Doppler ambiguity cuts of waveforms optimized with and without a spectral notch constraint (forbidding transmission in a band centered at 25% of the bandwidth). With the mask, energy in the forbidden band drops from 7.7% to below 0.01% at a cost of only 0.2 dB in PSL ($-34.8 \rightarrow -34.6$ dB), demonstrating that spectral avoidance can be achieved with negligible impact on ambiguity performance.

F. Neural Waveform Generator

1) Training: Fig. 5 shows the training convergence of the neural waveform generator for $N = 256$. The network converges within approximately 150 epochs (141 seconds on a Tesla T4 GPU with 10 gradient steps per epoch at batch size 32), after which it can generate waveforms for any λ value without further optimization.

2) Generated Waveform Quality: Table VII compares the quality of neural-generated waveforms against iteratively optimized waveforms across λ values, with error bars from 10 independent training seeds. The neural generator achieves PSL within 0.3–1.6 dB of the iterative optimizer. The gap is expected: the iterative method optimizes each waveform individually for 2000 iterations, while the generator must learn a single mapping that covers the entire λ range. The gap narrows at higher λ values, where the spectral variance objective smooths the loss landscape.

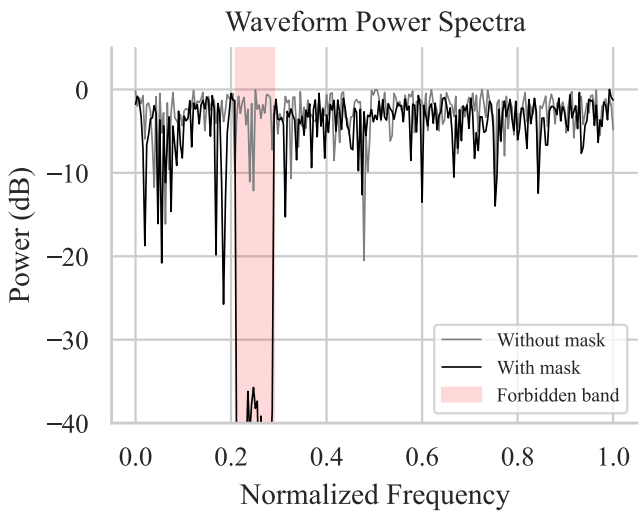

Fig. 4. Waveform power spectra with and without the spectral mask constraint. The forbidden band (red shading) is fully suppressed at a cost of only 0.2 dB in PSL ($-34.8 \rightarrow -34.6$ dB).

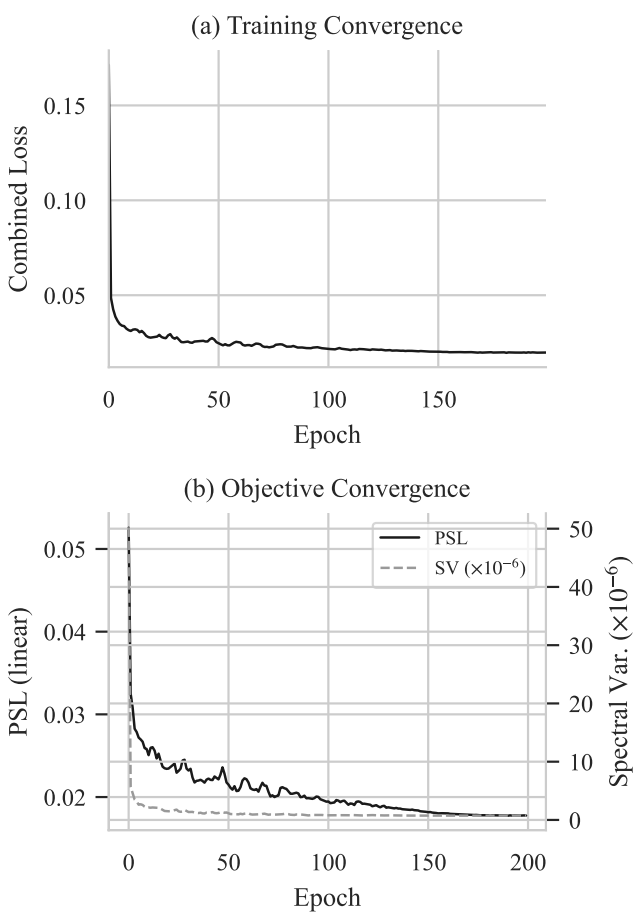

Fig. 5. Neural waveform generator training convergence ($N = 256$, Tesla T4). (a) Combined loss. (b) PSL and spectral variance objectives over training epochs.

3) Architecture Ablation: Table VI shows the effect of network size and training duration on generated waveform quality, using a reduced training budget (1 gradient step per epoch, batch size 16) to isolate architectural effects. Under this budget, the medium architecture (128 hidden, 3 layers) achieves the best PSL, suggesting that larger models require more training to realize their capacity. With the full training schedule (10 steps per epoch, batch size 32), the 256-hidden model achieves -34.2 dB (Table VII), outperforming the ablation results. Training for 200 epochs provides approximately 1 dB improvement over 50 epochs across all architectures.

TABLE VI
NEURAL GENERATOR ABLATION ($N = 256$, PSL IN DB, MEAN OVER 5 SEEDS).

Configuration	**Params**	λ=0	λ=1
Small (64d, 2L)	16,960	-31.5	-32.4
Medium (128d, 3L)	67,200	-32.6	-33.3
Full (256d, 4L)	332,288	-31.8	-32.4
Large (512d, 4L)	1,188,608	-30.7	-31.8
Full, 50 epochs	332,288	-30.7	-31.3
Full, 100 epochs	332,288	-31.4	-31.9
Full, 200 epochs	332,288	-31.8	-32.4

TABLE VII
NEURAL GENERATOR VS. ITERATIVE GRADIENT OPTIMIZATION ($N = 256$, MEAN $\pm$ STD OVER 10 SEEDS FOR NEURAL, 20 SEEDS FOR GRADIENT).

λ	**Neural PSL (dB)**	**Gradient PSL (dB)**
0.00	-33.7 ± 0.1	-35.3 ± 0.1
0.25	-34.1 ± 0.1	-35.1 ± 0.1
0.50	-34.1 ± 0.1	-34.8 ± 0.1
1.00	-34.2 ± 0.1	-34.5 ± 0.1
2.00	-34.2 ± 0.1	-34.1 ± 0.1

4) Inference Speed: The trained neural generator produces waveforms in a single forward pass. Table VIII compares the time to produce one optimized waveform across all three methods.

TABLE VIII
TIME TO PRODUCE ONE OPTIMIZED WAVEFORM ($N = 256$, TESLA T4).

Method	**Time**	**Requires optimization?**
GA	$\sim$14.4 s	Yes (per waveform)
Gradient descent	$\sim$7.0 s	Yes (per waveform)
Neural generator	$<$1 ms	No (single forward pass)

The neural generator achieves a $>10{,}000\times$ speedup at inference time compared to gradient descent, and $>20{,}000\times$ compared to GA. The one-time training cost (141 seconds) is amortized across all subsequent waveform generation requests—equivalent to optimizing approximately 20 waveforms iteratively.

G. Scaling and GPU Performance

1) Waveform Length Scaling: Fig. 6 shows computation time and PSL performance as a function of waveform length

N. PSL improves by approximately 6 dB per doubling of N, consistent with theoretical expectations.

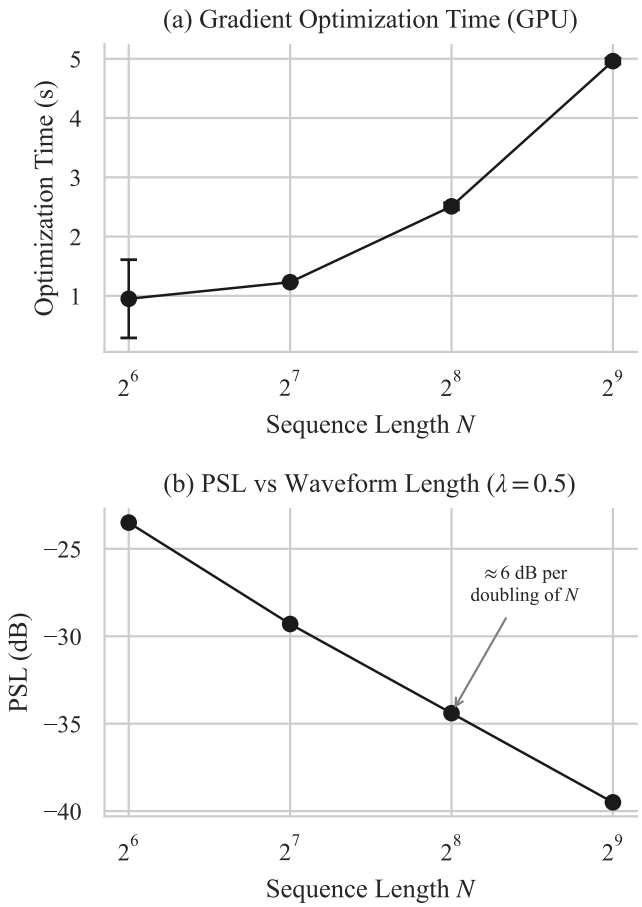

Fig. 6. Scaling with waveform length N (GPU, $\lambda = 0.5$). (a) Optimization time. (b) Achieved PSL, showing ≈6 dB improvement per doubling of N.

2) GPU Acceleration: Table IX reports GPU vs. CPU performance for the differentiable ambiguity function forward pass.

TABLE IX
GPU VS. CPU FORWARD PASS TIME (MILLISECONDS).

N	**CPU**	**GPU**	**Speedup**
64	0.25	0.23	1.1×
128	0.56	0.21	2.6×
256	1.68	0.22	7.8×
512	8.65	0.22	39.0×
1024	32.57	0.86	37.9×

GPU acceleration provides increasing benefit for larger N, as shown in Fig. 7, with speedups reaching $39\times$ at $N = 512$. The GPU forward pass time remains nearly constant (≈0.2 ms) up to $N = 512$, while CPU time scales quadratically, demonstrating the parallelism of the matrix-based formulation.

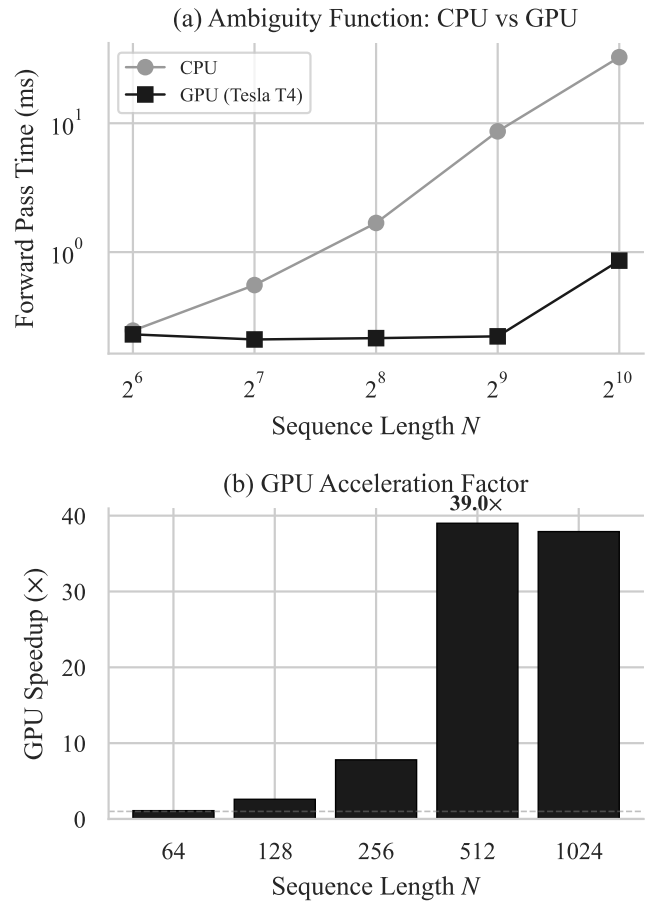

Fig. 7. GPU vs. CPU performance for the differentiable ambiguity function forward pass (Tesla T4). (a) Absolute timing. (b) GPU acceleration factor, reaching $39\times$ at $N = 512$.

VI. Discussion

A. Practical Implications

The neural waveform generator supports a practical operating mode for cognitive radar systems. Rather than running iterative optimization in the radar processing loop—which is computationally prohibitive for real-time adaptation—a pre-trained generator can produce optimized waveforms instantly in response to changing environmental conditions or mission requirements.

The differentiable ambiguity function also benefits traditional iterative optimization workflows. Adding a new performance metric to the optimization (e.g., Doppler tolerance, spectral mask compliance, interference mitigation) requires only defining the metric as a differentiable function—no gradient re-derivation. This lowers the barrier for radar engineers to experiment with different multi-objective formulations.

B. Comparison to Analytical Gradient Methods

We do not claim that our differentiable implementation produces better gradients than the analytical derivations of Alhujaili et al. [8] or Mohr et al. [9] for their specific problem formulations. For a fixed objective with known analytical gradients, a specialized implementation may be more

computationally efficient, as it avoids the overhead of AD graph construction. Direct comparison is not straightforward, as these methods address different formulations: QGD [8] operates on the slow-time ambiguity function under constant modulus constraints on the complex circle manifold, while Mohr et al. [9] target PCFM waveforms specifically.

Our contribution is *generality* and *composability*. The analytical methods derive gradients for specific cost functions on specific manifolds; our implementation provides gradients for any differentiable function of the periodic ambiguity surface considered here, and can be composed with upstream neural networks and downstream custom loss terms. The neural waveform generator is the clearest demonstration of this composability: reproducing the same end-to-end training workflow with analytical methods would require deriving and maintaining gradients through the full ambiguity computation for each new upstream parameterization.

C. Limitations

Periodic assumption. Our formulation computes the periodic (circular) ambiguity function, appropriate for phase-coded waveforms but not directly applicable to all waveform types. Extension to aperiodic formulations requires alternative correlation computation strategies while maintaining differentiability.

Memory scaling. The $O(N^2)$ memory requirement for the circulant shift matrix scales quadratically; however, even $N = 16{,}384$ requires only $\sim$6 GB with gradient buffers, which is practical on current GPUs. For significantly longer waveforms, block-based or sparse formulations could reduce memory usage.

Generator generalization. The neural generator's quality is bounded by its training distribution. Extrapolation beyond the training range of λ values may degrade performance. However, retraining with an expanded range is straightforward given the differentiable pipeline.

Baseline scope. We compare against GA as the multi-objective baseline rather than reimplementing the specific gradient methods of [8] or [9], as those address different problem formulations (slow-time ambiguity shaping and PCFM waveforms, respectively) that are not directly comparable to our phase-coded PSL/LPI setup.

D. Future Work

All experiments in this work are simulation-based. Hardware-in-the-loop validation—measuring the actual ambiguity function of generated waveforms on a software-defined radio (SDR) testbed—would further strengthen the practical claims and is a natural next step enabled by the open-source implementation.

Other extensions include cross-ambiguity functions for MIMO radar, more expressive generator architectures (e.g., conditional transformers), integration with full radar signal processing chains for end-to-end system optimization, and deployment on embedded radar platforms with fixed-point constraints.

VII. Conclusion

We have presented a practical AD-compatible formulation of the periodic radar ambiguity function and shown that it enables both flexible full-2D waveform optimization and end-to-end neural waveform generation. The formulation uses parallelized FFT operations with proper Wirtinger derivative handling, achieving $O(N^2 \log N)$ complexity while maintaining mathematical equivalence to the standard periodic ambiguity function.

We demonstrated two applications of this differentiable implementation: (i) gradient-based optimization on the full 2D ambiguity function, achieving 2.2–3.0 dB PSL improvement over GA and 7.3 dB over 1D autocorrelation baselines; (ii) extensibility to new constraints without gradient re-derivation, demonstrated by adding a spectral mask objective; and (iii) a neural waveform generator that, once trained in approximately 141 seconds on GPU, produces optimized waveforms (within 0.3–1.6 dB of iterative optimization) for any design specification in under one millisecond—a capability enabled by the AD-compatible ambiguity function.

The implementation is open-source, GPU-accelerated, and compatible with standard deep learning frameworks, providing a bridge between classical radar signal processing and modern machine learning.

Acknowledgment

GPU experiments were conducted using Modal cloud compute.

AI Disclosure: An AI assistant (Claude, Anthropic) was used during the preparation of this manuscript for code development assistance, LaTeX editing, and prose refinement. All technical content, experimental design, and scientific conclusions are the sole responsibility of the author. The AI system was not used to generate figures, experimental results, or mathematical derivations.

References

[1] P. M. Woodward, *Probability and Information Theory, with Applications to Radar*. London: Pergamon Press, 1953.
[2] N. Levanon and E. Mozeson, *Radar Signals*. John Wiley & Sons, 2004.
[3] S. D. Blunt and K. Gerlach, "Overview of radar waveform diversity," *IEEE Aerosp. Electron. Syst. Mag.*, vol. 31, no. 11, pp. 2–42, 2016.
[4] G. U. Kim and J. P. Kim, "Modified Price nonlinear frequency modulated waveform for improved performance," *IEEE Signal Process. Lett.*, 2025.
[5] S. Maresca, A. Malacarne, M. M. H. Amir, F. Ahmad, G. Pandey, A. Bogoni, and M. Scaffardi, "Genetic algorithms for distributed MIMO radar antenna position optimization," in *Proc. 27th Int. Conf. Inf. Fusion (FUSION)*, 2024.
[6] K. Farnane and K. Minaoui, "Optimization of a new Golay sequences shaping for low sidelobe radar ambiguity function," *Signal, Image Video Process.*, 2020.
[7] B. Niu, Y. Zhao, M. Zhang, D. Tang, T. Zhang, S. Zhang, and D. Gao, "A novel energy-focused slow-time MIMO radar and signal processing scheme," *Signal Processing*, 2025.
[8] K. Alhujaili, V. Monga, and M. Rangaswamy, "Quartic gradient descent for tractable radar slow-time ambiguity function shaping," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 56, no. 4, pp. 2447–2464, 2020.
[9] C. A. Mohr, P. M. McCormick, C. A. Topliff, S. D. Blunt, and J. M. Baden, "Gradient-based optimization of PCFM radar waveforms," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 57, no. 4, pp. 2049–2064, 2021.
[10] J. M. Baden, B. O'Donnell, and L. Schmieder, "Multiobjective sequence design via gradient descent methods," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 54, no. 3, pp. 1237–1252, 2018.

[11] A. Aubry, A. De Maio, B. Jiang, and S. Zhang, "Ambiguity function shaping for cognitive radar via complex quartic optimization," *IEEE Trans. Signal Process.*, vol. 61, no. 22, pp. 5603–5619, 2013.

[12] M. Alaee-Kerahroodi, A. Aubry, A. De Maio, M. M. Naghsh, and M. Modarres-Hashemi, "A coordinate-descent framework to design low PSL/ISL sequences," *IEEE Trans. Signal Process.*, vol. 65, no. 22, pp. 5942–5956, 2017.

[13] J. Song, P. Babu, and D. P. Palomar, "Optimization methods for designing sequences with low autocorrelation sidelobes," *IEEE Trans. Signal Process.*, vol. 63, no. 15, pp. 3998–4009, 2015.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[15] S. Haykin, "Cognitive radar: A way of the future," *IEEE Signal Process. Mag.*, vol. 23, no. 1, pp. 30–40, 2006.

[16] S. Z. Gurbuz, H. D. Griffiths, A. Charlish, M. Rangaswamy, E. Mokole, and K. Bell, "An overview of cognitive radar: Past, present, and future," *IEEE Aerosp. Electron. Syst. Mag.*, vol. 34, no. 12, pp. 6–18, 2019.

[17] B. Kang, J. Kweon, M. Rangaswamy, and V. Monga, "Deep learning for radar waveform design: Retrospectives and the road ahead," in *Proc. IEEE Radar Conf.*, 2023.

[18] M. R. Ziemann and C. A. Metzler, "Adaptive LPD radar waveform design with generative adversarial neural networks," in *Proc. Asilomar Conf. Signals, Syst. Comput.*, 2023.

[19] A. L. Sénica, P. A. C. Marques, and M. A. T. Figueiredo, "Artificial intelligence applications in noise radar technology," *IET Radar, Sonar Navig.*, vol. 18, no. 7, pp. 986–1001, 2024.

[20] O. Rezaei, M. Ahmadi, M. M. Naghsh, A. Aubry, M. M. Nayebi, and A. De Maio, "A learning-inspired strategy to design binary sequences with good correlation properties: SISO and MIMO radar systems," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 6, pp. 8985–8997, 2023.

[21] J. Yang, X. Gu, W. Li, Y. Qiu, Z. Sun, and J. Wu, "Configuration design of bistatic forward-looking SAR driven by spatial resolution metrics," *IEEE Geosci. Remote Sens. Lett.*, 2025.

[22] Y. Wang, Y. Cao, T.-S. Yeo, J. Han, and Z. Peng, "Joint sequence optimization-based OFDM waveform design for integrated radar and communication systems," *IEEE Trans. Veh. Technol.*, 2022.

[23] Y. Jing *et al.*, "Designing unimodular sequence with low peak of sidelobe level of local ambiguity function," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 55, no. 3, pp. 1393–1406, 2019.

[24] K. Alhujaili, "Non-convex optimization techniques for radar waveform design under practical constraints," Ph.D. dissertation, Pennsylvania State University, 2020.

[25] X. Xiao, H. Wang, J. Hu, X. Tai, Y. Zuo, H. Li, K. Zhong, and D. An, "Unimodular waveform design for ambiguity function shaping with spectral constraint via a manifold-based exact penalty method," *Signal Processing*, 2025.

[26] X. Qiu, W. Jiang, Y. Liu, S. Chatzinotas, F. Gini, and M. S. Greco, "Constrained Riemannian manifold optimization for the simultaneous shaping of ambiguity function and transmit beampattern," *IEEE Trans. Aerosp. Electron. Syst.*, 2025.

[27] G. Reddy, J. Pandu, and C. Babu, "Optimizing FECAF in MTSFM waveforms using conjugate gradient descent with efficient line search for radar and sonar applications," *Int. J. Commun. Syst.*, 2025.

[28] N. Del-Rey-Maestre, M. P. Jarabo-Amores, D. Mata-Moya, and J. Rosado-Sanz, "Decentralised intelligent passive radar detection based on clutter modelling," in *Proc. 21st European Radar Conf. (EuRAD)*, 2024.

[29] D. B. Herr, P. S. Raju, and J. M. Stiles, "Information theoretic waveform design with applications to adaptive-on-transmit radar," *IET Radar, Sonar Navig.*, vol. 18, no. 1, pp. 222–234, 2024.

[30] M. Bara Iniesta, "Differentiable radar ambiguity functions: Mathematical formulation and computational implementation," *arXiv preprint arXiv:2506.22935*, 2025.

[31] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: A survey," *J. Mach. Learn. Res.*, vol. 18, no. 153, pp. 1–43, 2018.

[32] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2015.